JavaScript is disabled on your browser.

compact1, compact2, compact3

java.lang

Class ClassValue<T>

java.lang.Object
    java.lang.ClassValue<T>

```
public abstract class ClassValue<T>
extends Object
```

Lazily associate a computed value with (potentially) every type. For example, if a dynamic language needs to construct a message dispatch table for each class encountered at a message send call site, it can use a `ClassValue` to cache information needed to perform the message send quickly, for each class encountered.

Since:

1.7

Constructor Summary

Constructors

Modifier	Constructor	Description
protected	`ClassValue()`	Sole constructor.

Method Summary

Abstract Methods

Concrete Methods

Modifier and Type	Method	Description

protected abstract T	**computeValue**(Class<?> type)	Computes the given class's derived value for this ClassValue.
T	**get**(Class<?> type)	Returns the value for the given class.
void	**remove**(Class<?> type)	Removes the associated value for the given class.

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Detail

ClassValue

```
protected ClassValue()
```

Sole constructor. (For invocation by subclass constructors, typically implicit.)

Method Detail

computeValue

```
protected abstract T computeValue(Class<?> type)
```

Computes the given class's derived value for this `ClassValue`.

This method will be invoked within the first thread that accesses the value with the `get` method.

Normally, this method is invoked at most once per class, but it may be invoked again if there has been a call to `remove`.

If this method throws an exception, the corresponding call to `get` will terminate abnormally with that exception, and no class value will be recorded.

Parameters:

`type - the type whose class value must be computed`

Returns:

the newly computed value associated with this ClassValue, for the given class or interface

See Also:

`get(java.lang.Class<?>)`, `remove(java.lang.Class<?>)`

get

```
public T get(Class<?> type)
```

Returns the value for the given class. If no value has yet been computed, it is obtained by an invocation of the `computeValue` method.

The actual installation of the value on the class is performed atomically. At that point, if several racing threads have computed values, one is chosen, and returned to all the racing threads.

The `type` parameter is typically a class, but it may be any type, such as an interface, a primitive type (like `int.class`), or `void.class`.

In the absence of `remove` calls, a class value has a simple state diagram: uninitialized and initialized. When `remove` calls are made, the rules for value observation are more complex. See the documentation for `remove` for more information.

Parameters:

`type - the type whose class value must be computed or retrieved`

Returns:

`the current value associated with this ClassValue, for the given class or interface`

Throws:

`NullPointerException - if the argument is null`

See Also:

`remove(java.lang.Class<?>)`, `computeValue(java.lang.Class<?>)`

remove

```
public void remove(Class<?> type)
```

Removes the associated value for the given class. If this value is subsequently read for the same class, its value will be reinitialized by invoking its `computeValue` method. This may result in an additional invocation of the `computeValue` method for the given class.

In order to explain the interaction between `get` and `remove` calls, we must model the state transitions of a class value to take into account the alternation between uninitialized and initialized states. To do this, number

these states sequentially from zero, and note that uninitialized (or removed) states are numbered with even numbers, while initialized (or re-initialized) states have odd numbers.

When a thread `T` removes a class value in state `2N`, nothing happens, since the class value is already uninitialized. Otherwise, the state is advanced atomically to `2N+1`.

When a thread `T` queries a class value in state `2N`, the thread first attempts to initialize the class value to state `2N+1` by invoking `computeValue` and installing the resulting value.

When `T` attempts to install the newly computed value, if the state is still at `2N`, the class value will be initialized with the computed value, advancing it to state `2N+1`.

Otherwise, whether the new state is even or odd, `T` will discard the newly computed value and retry the `get` operation.

Discarding and retrying is an important proviso, since otherwise `T` could potentially install a disastrously stale value. For example:

- `T` calls `CV.get(C)` and sees state `2N`
- `T` quickly computes a time-dependent value `V0` and gets ready to install it
- `T` is hit by an unlucky paging or scheduling event, and goes to sleep for a long time
- ...meanwhile, `T2` also calls `CV.get(C)` and sees state `2N`
- `T2` quickly computes a similar time-dependent value `V1` and installs it on `CV.get(C)`
- `T2` (or a third thread) then calls `CV.remove(C)`, undoing `T2`'s work
- the previous actions of `T2` are repeated several times
- also, the relevant computed values change over time: `V1`, `V2`, ...
- ...meanwhile, `T` wakes up and attempts to install `V0`; *this must fail*

We can assume in the above scenario that `CV.computeValue` uses locks to properly observe the time-dependent states as it computes `V1`, etc. This does not remove the threat of a stale value, since there is a window of time between the return of `computeValue` in `T` and the installation of the the new value. No user synchronization is possible during this time.

Parameters:

`type - the type whose class value must be removed`

Throws:

C

`NullPointerException` - if the argument is null

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.